

VELOCEAI · WISP GUIDE

Make your own living guide

Wisp is a small animated character that welcomes your visitors, shows them around, answers their questions in their own language, reads the page, and helps them get things done. It works on any website, inside React, Next.js and Vue apps, and in desktop apps.

This guide shows how to add it in one line, give it your own look, and teach it about your product.

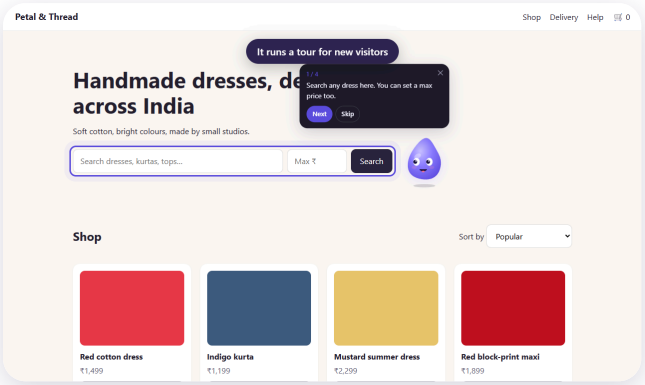


Wisp · Ember · Sprout · Bolt · Mochi

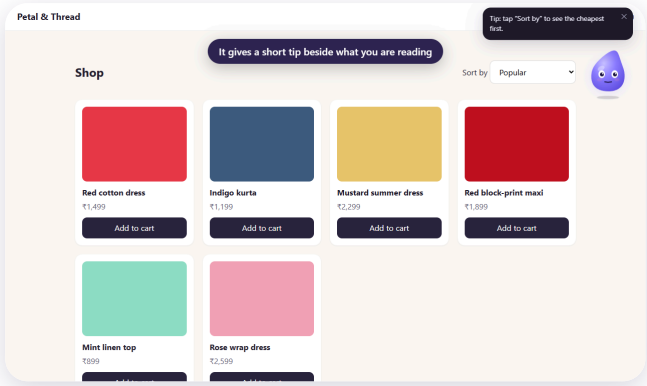
MIT licence · our own code and art

WHAT IT DOES

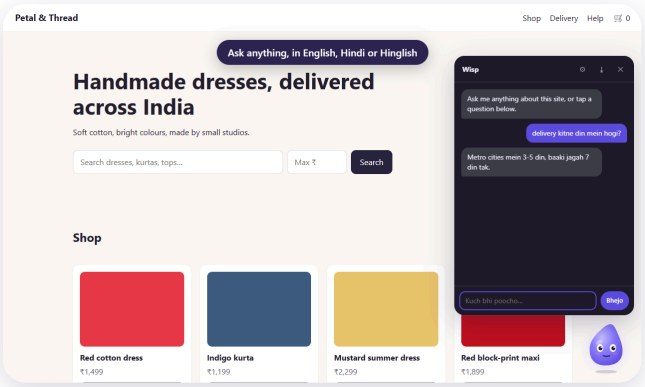
A guide that feels alive, and actually helps



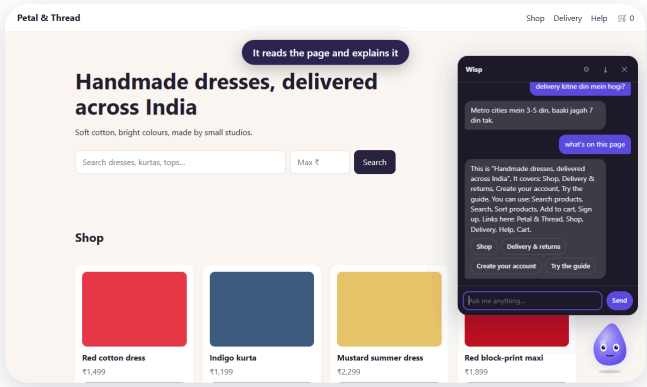
Tours. New visitors get a short walkthrough, step by step.



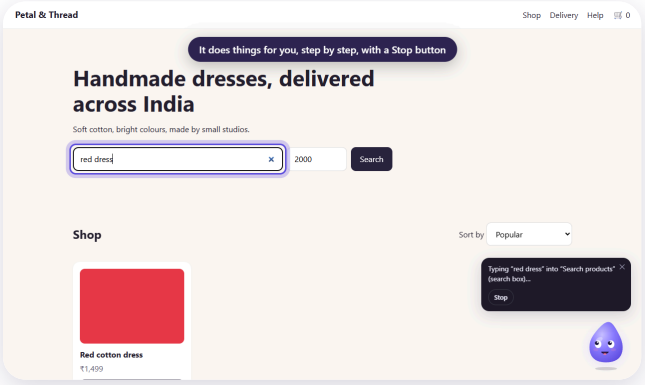
Tips. One short tip beside the section being read.



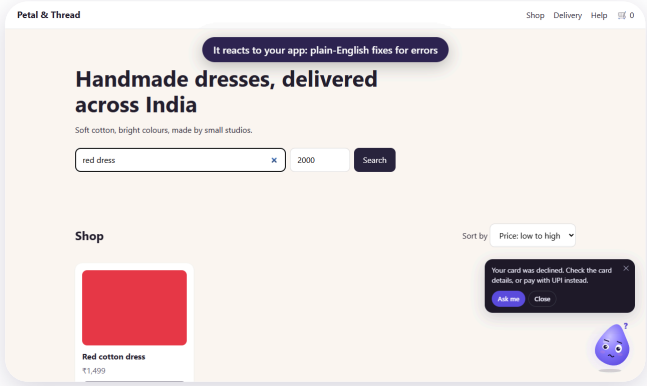
Chat. Answers from your FAQ and pages, in English, Hindi, Hinglish or any language you add.



Understands the page. Explains what's here and points at things.



Does things. "Search red dress under ₹2000 and sort by price", step by step, with a Stop button.

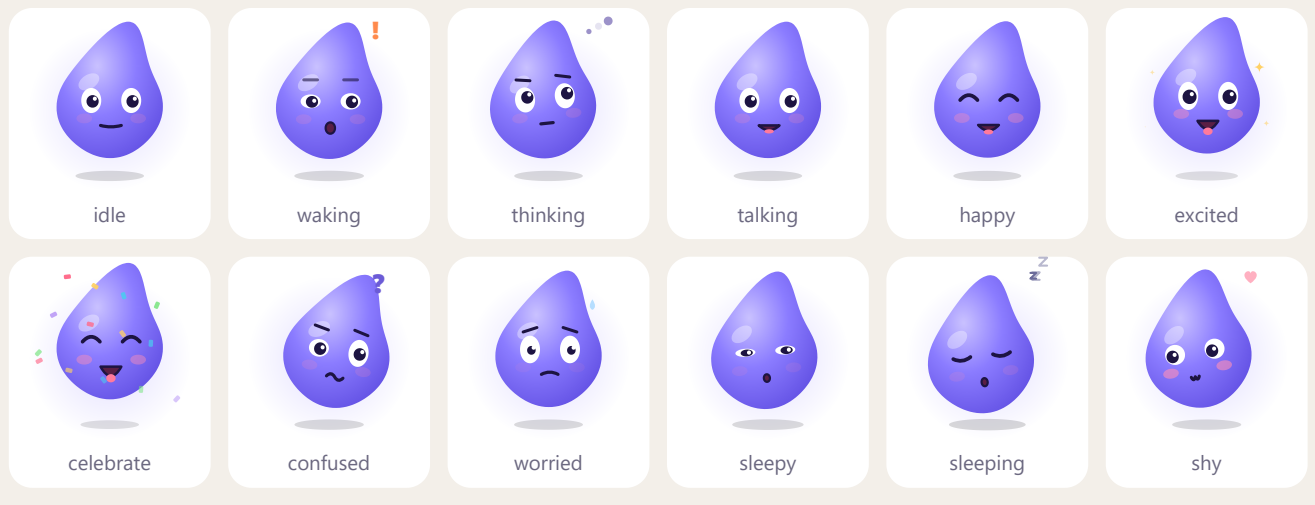


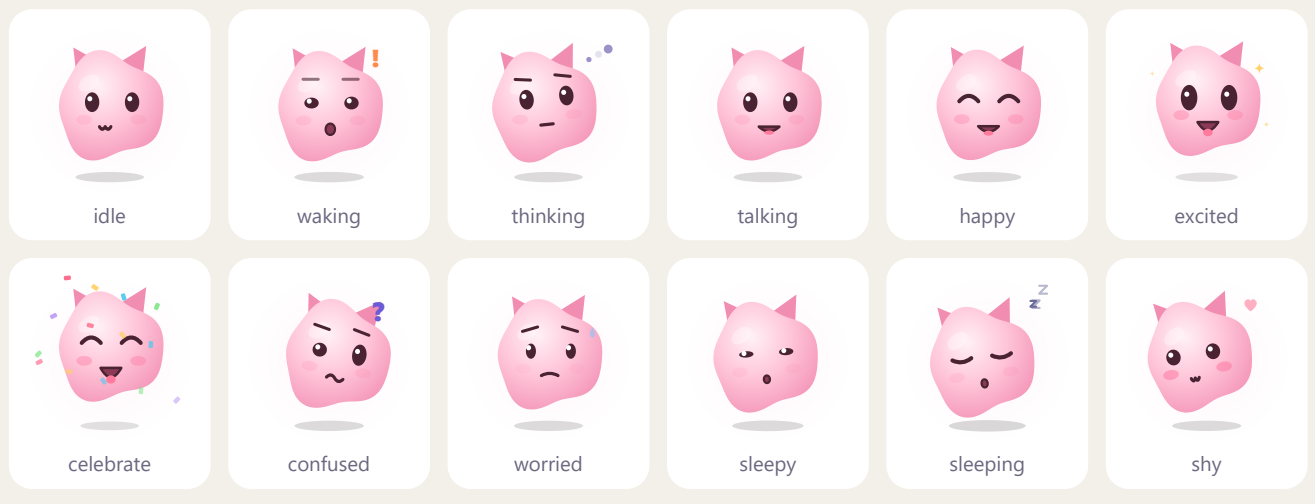
Reacts. Celebrates success, explains errors in plain English.

THE CHARACTER

12 moods, any look

Every character is drawn by Wisp itself from a short description, so it stays sharp at any size and every mood works with every look. It blinks, breathes, follows the cursor, giggles when tapped, and falls asleep when the visitor is idle. People who turn on "reduce motion" get a calm, still version.





MOOD	WHEN IT SHOWS
waking, happy	The visitor arrives, taps it, or comes back
talking, thinking	It speaks; it's working something out or the page is loading
excited, celebrate	A goal is reached: order placed, account created
confused, worried	An error, or before something final like paying
sleepy, sleeping, shy, idle	Nobody's around, it's tapped a lot, or it's just waiting

STEP 1 ADD IT

One line on any website

```
<script src="/wisp.js" data-config="/wisp.json" defer></script>
```

`wisp.js` is built to `packages/web/dist/wisp.js` (about 22 KB zipped). Put it next to your site and write `wisp.json` (step 3). That's all; it works with no AI.

React and Next.js

```
import { Wisp } from '@veloceai/wisp-react';
const router = useRouter();

<Wisp config={config} navigate={(path) => router.push(path)}>
  <App />
</Wisp>
```

Vue

```
<Wisp :config="config"><App /></Wisp>   // from '@veloceai/wisp-vue'
```

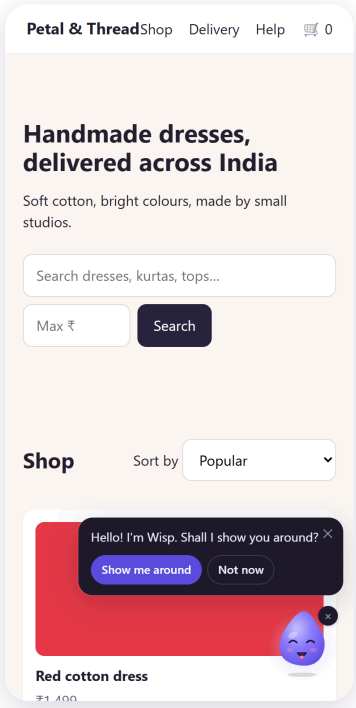
Desktop apps

Electron and Tauri apps use the web build as is. Keep any AI key in the system keychain, never in the app's web code.

Phones

On phones Wisp stays small in a bottom corner and moves away if it would cover a button. If your app has a bottom tab bar, tell it how tall: `"insets": { "bottom": 64 }`.

A small × hides it into a tab at the edge; it remembers that.



STEP 2 YOUR OWN CHARACTER

Describe it, Wisp draws it

```
"character": {
  "name": "Tara",
  "body": {
    "shape": "blob",
    "color": "#FF8A4C",
    "shade": "#E2552B",
    "glow":  "#FFD0A8"
  },
  "eyes":  { "style": "sparkle" },
  "mouth": { "style": "cat" },
  "accessory": {
    "kind": "antenna",
    "color": "#FFD166"
  }
}
```

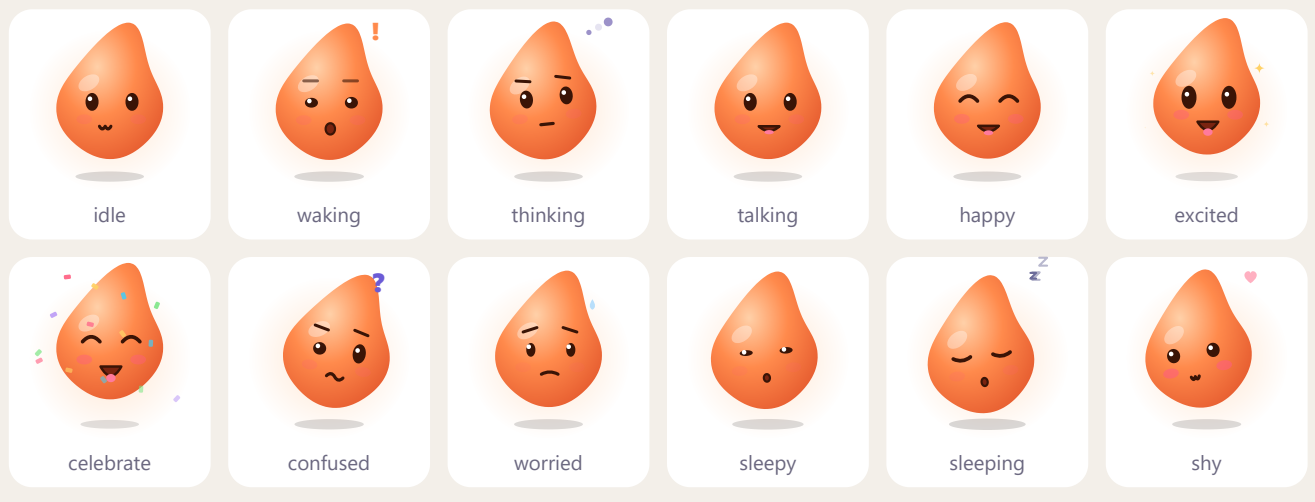
PART	CHOICES
shape	wisp, round, squircle, blob, bean
eyes	classic, dot, sparkle · size, spacing, height
mouth	smile, cat, tiny
accessory	none, antenna, sprout, ears, crown
colours	body, shade, glow, belly, outline, cheeks

Or start from a sample: `"character": "ember"`.

Size and style

```
"size":  { "desktop": "large", "phone": "small" },     // or a number of pixels
"theme": { "accent": "#E2552B", "font": "Poppins, sans-serif", "radius": 8, "mode": "auto"
}
```

Changing the look never changes behaviour. Wisp lives in its own walled-off layer, so your site's styles can't break it and it can't break your site.



STEP 3 TEACH IT YOUR PRODUCT

Write what it should say

```
{
  "siteId": "my-shop",
  "tour": [
    { "target": "#search", "text": "Search any dress here." },
    { "target": "#sort",   "text": { "en": "Sort by price.", "hinglish": "Price se sort
karo." } }
  ],
  "sections": [ { "id": "pricing", "tip": "The free plan covers one project." } ],
  "faq": [
    { "q": ["How long is delivery?", { "hinglish": "Delivery kitne din mein?" }],
      "a": { "en": "3 to 5 days.", "hi": "3 से 5 दिन।", "hinglish": "3 se 5 din." } }
  ],
  "errors":    [ { "match": "card_declined", "fix": "Your card was declined. Try UPI
instead." } ],
  "reactions": { "cart-add": { "mood": "happy", "line": "Nice pick!" } }
}
```

- Mark sections with `data-wisp-section="pricing"` so it knows where tips belong.
- Tell it what happens in your app with window events: `wisp:success`, `wisp:error`, `wisp:loading`, `wisp:loaded`, `wisp:conversion`, or any name under "reactions".
- Any language: write `{ "ta": "…" }` and list it under `"languages"`. Button labels go under `"ui"`.
- New visitors get step-by-step help; regulars get short advanced tips. Visitors can switch between "Help me a lot" and "Just tips".

STEP 4 OPTIONAL EXTRAS

Your own AI key, voice, and actions

AI from any provider

OpenAI, Anthropic, Gemini, Groq, OpenRouter or a local Ollama. The key stays on your server in a tiny proxy; visitors' browsers never see it.

```
WISP_PROVIDER=gemini WISP_API_KEY=… WISP_ORIGINS=https://myshop.in npx wisp-proxy
// then in wisp.json:  "ai": { "endpoint": "https://myshop.in/wisp" }
```

Built-in cost limits: 20 AI calls per visitor a day, an 8-second timeout, and prewritten answers whenever the AI is off, slow or over its limit. Questions your FAQ already answers never call the AI.

Voice (off until the visitor turns it on)

The device's own voice for free, or a natural AI voice through the proxy. Each line is spoken once and then reused at no cost. A mic button lets visitors ask out loud.

Let it do things

```
"actions": [
  { "name": "search", "description": "Search products", "params": { "query": { "type":
"string" } } },
  { "name": "track-order", "description": "Track my order", "button": "Track my order" },
  { "name": "place-order", "description": "Place the order", "final": true }
]
Wisp.instance.registerAction('search', ({ query }) => runMySearch(query));
```

SAFETY

Rules it always follows

- **Asks before anything final:** pay, buy, place order, delete, send, submit, sign up, log out, or leaving your site.
- **Never touches private fields:** passwords, card numbers, OTP, CVV, PIN and ID numbers are never read, typed into, or sent to an AI. Mark anything else with `data-wisp-sensitive`.
- **Stays out of blocked areas:** `data-wisp="off"` on an element, or `"blocked": { "selectors": [".admin"], "paths": ["/admin*"] }`.
- **Keys stay on the server:** websites use the proxy; desktop apps use the system keychain.
- **Always stoppable:** a Stop button stays up while it acts.
- **Polite:** one unasked tip at a time, at most every 8 seconds, each tip once, and it waits while your own dialogs are open.

Checklist before you go live

☐	Welcome and tour work on your home page
☐	Each key section has a short tip
☐	10+ real FAQ answers, in the languages your visitors use
☐	Success and error events fire at the right moments
☐	Private areas are blocked; private inputs are marked
☐	On a phone it doesn't cover your buttons or tab bar
☐	If AI is on: the key is only on the server and origins are set

Full reference, a demo shop and a one-minute video: **github.com/VeloceAI/wisp**.

